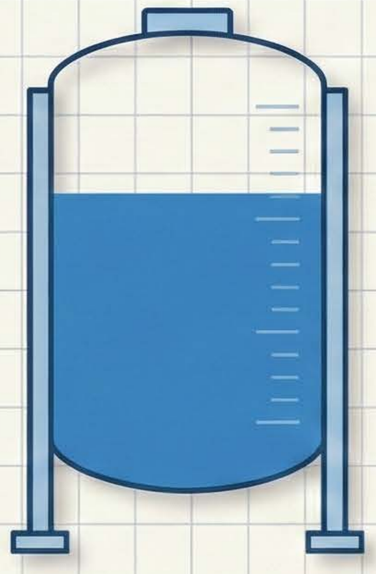
控制流程 (Control Flow)：打造數位化工廠的神經系統

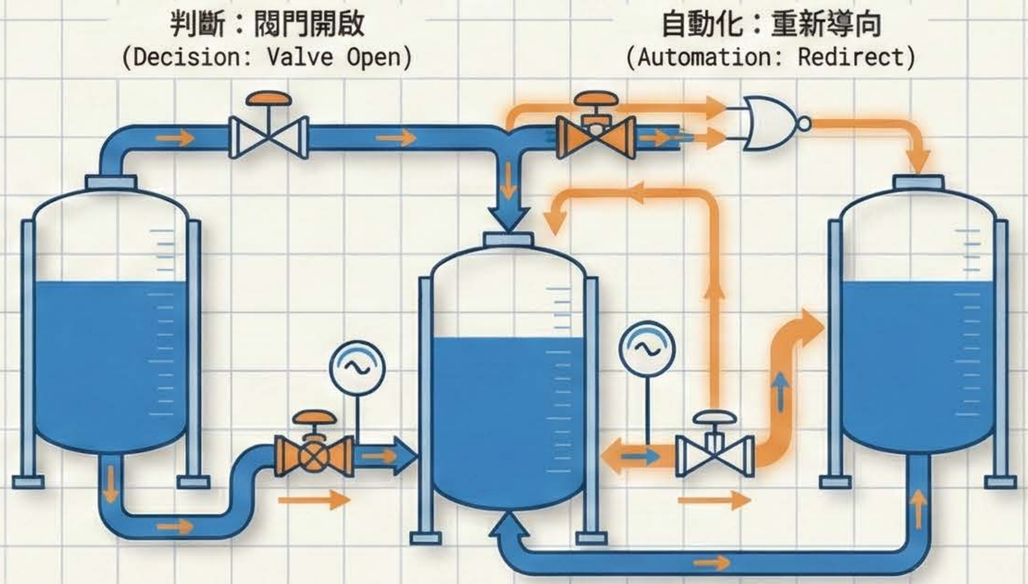
Unit 02 Python 基礎教學 - Section 04

變數 (Variables)



儲存原料 (Raw Material Storage)

控制流程 (Control Flow)



管線邏輯與閥門 (Pipeline Logic & Valves)

- ✅ 變數 (Variables) 是儲存原料的容器。
- ✅ 控制流程 (Control Flow) 是決定原料流向的管線邏輯與閥門。
- ✅ 本章節重點：賦予程式「判斷」與「自動化」的能力。

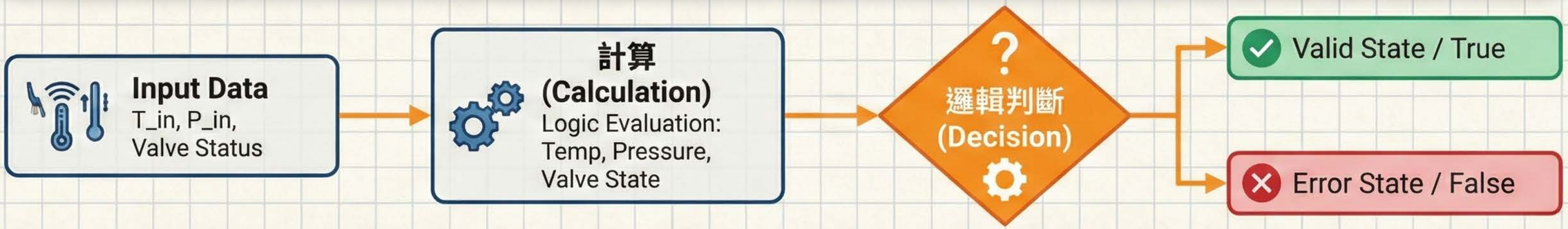
適用對象：化工工程師、資料科學家

NotebookLM

決策的基礎：邏輯與比較運算子

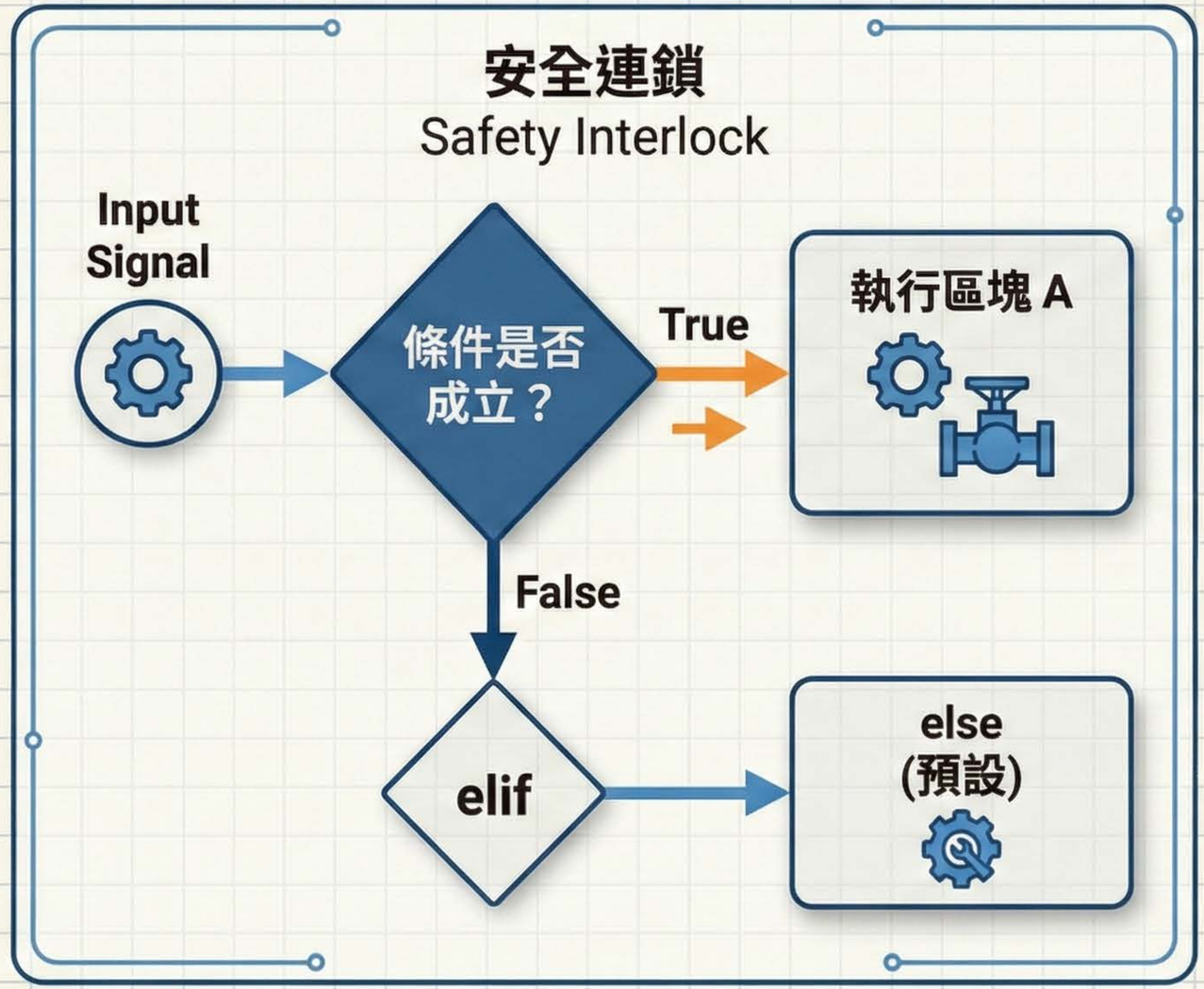
在化工廠中，閥門的開關取決於感測器的數值（如：溫度 > 100）。在 Python 中，這稱為布林邏輯（Boolean Logic）。

比較 (Comparison)	邏輯 (Logical)
`==` (等於) `!=` (不等於) `>` (大於) `<` (小於)	**and** (且)：兩者皆成立 `P > 50 and Valve == 'Open'`
`Temp > 100` → Returns True / Returns False	**or** (或)：其中之一成立 `T > 150 or P > 200`
	not (非)：反轉狀態 `not Emergency`



NotebookLM

條件語句（Conditions）：程式碼的安全連鎖系統



Python Syntax (程式語法)

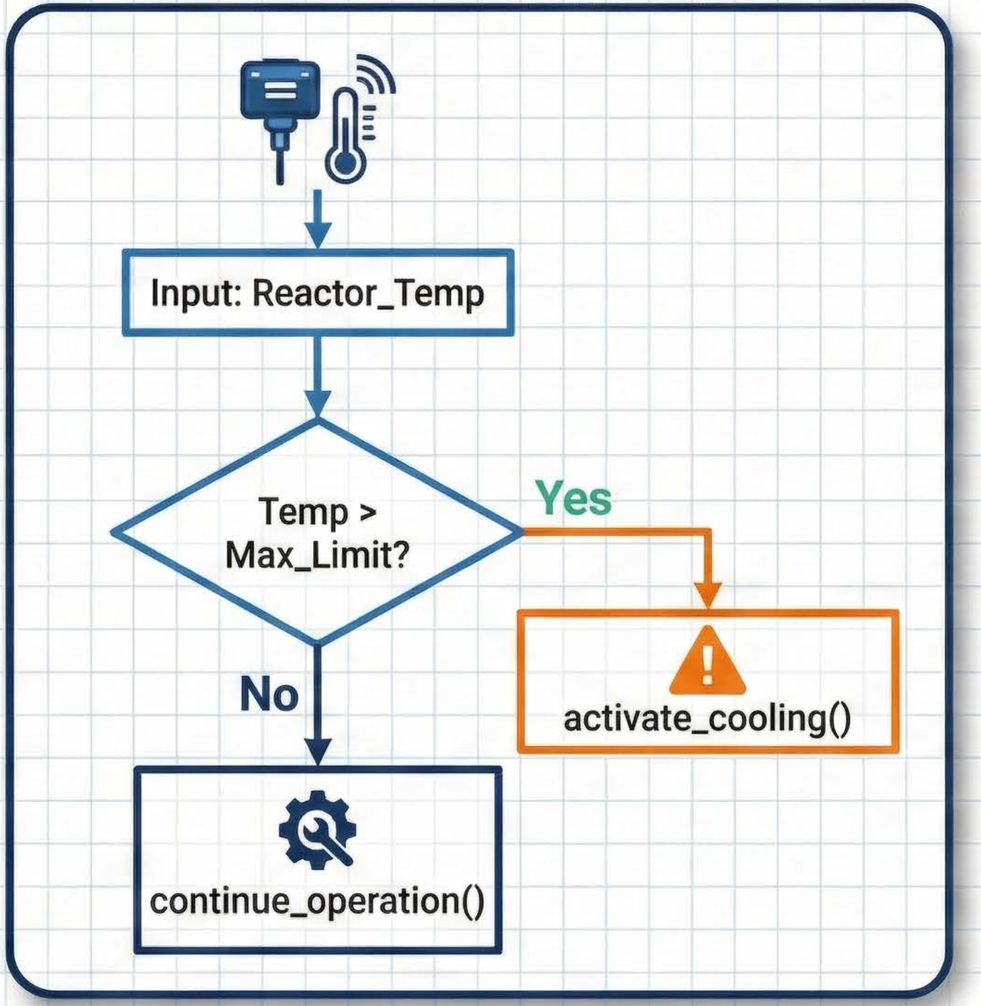
```
if condition:
    # 必須縮排 (Indentation)
elif another_condition:
    # 可選路徑
else:
    # 最後的預設方案
```

4-space indentation (必須縮排)

縮排 (Indentation) 是 Python 區分程式區塊的唯一方式，如同實體管線的隔離區。

NotebookLM

實戰案例：反應器操作監控




```
if temperature > 150:
    status = 'Critical'   # 危險：溫度過高
    action = 'Shutdown'
elif temperature > 120:
    status = 'Warning'    # 警告：需注意
    action = 'Cooling'
else:
    status = 'Normal'     # 正常運作
    action = 'Monitor'
```

Primary Interlock (主連鎖)

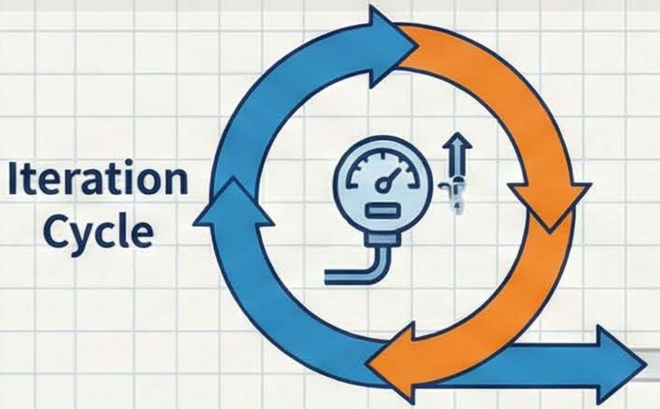
Secondary Warning (次級警告)

迴圈 (Loops)：自動化批次處理

工程師不會手動逐一檢查 100 個儀表；我們使用自動化程序掃描整個清單。

Manual (人工操作)

```
print(sensor_1)
print(sensor_2)
print(sensor_3)
...
```



Automated (自動化)

```
for sensor in all_sensors:
    print(sensor)
```

FIXED COUNT

For 迴圈 (For Loop)

已知次數的重複（巡檢儀表板上的每一個儀表）。

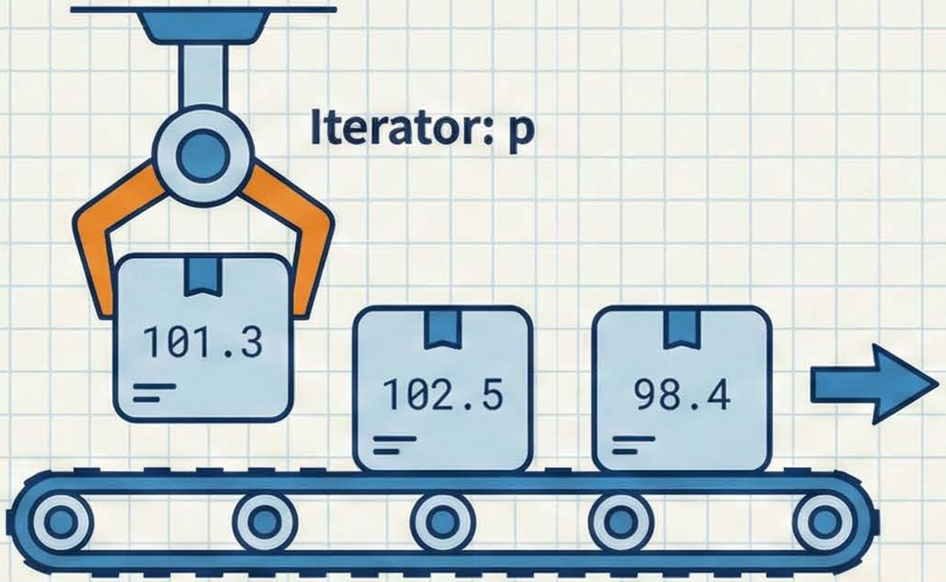
CONDITION-BASED

While 迴圈 (While Loop)

執行直到條件滿足（持續加熱直到溫度達標）。

For 迴圈：定序巡檢 (Sequential Iteration)

用於遍歷序列（如 List, String, Range）中的每個元素。




```python
pressures = [101.3, 102.5, 98.4, 105.1]

for p in pressures:
    print(f'Current Pressure: {p} kPa')
```

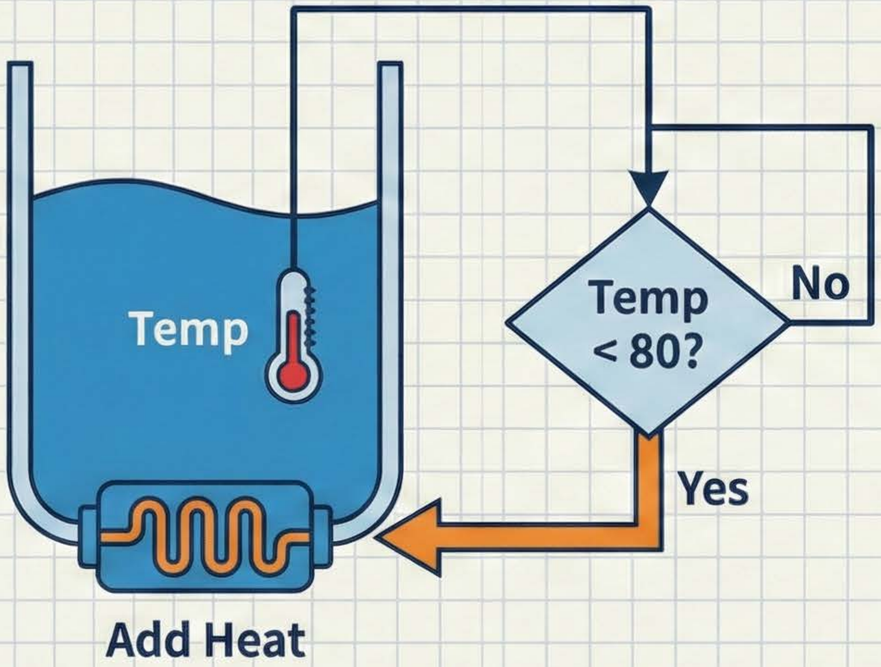
- Iterable (序列)
- Iterator (暫存變數)
- Execution Block (執行區塊)

Pro Tip: `range(start, stop)` 常用於產生數字序列（例如：執行 10 次模擬）。

NotebookLM

While 迴圈：條件控制 (Condition-Based)

只要條件為 True，就持續執行。類似化工製程中的「回饋控制」。




```python
temp = 25
target = 80

while temp < target:
    temp += 5  # 加熱
    print(f'Heating... Current Temp: {temp}°C')
```

SAFETY WARNING

無限迴圈（Infinite Loop）風險

如果忘記更新條件變數（如 temp += 5），程式將永遠跑下去，導致系統當機 (System Crash)。

迴圈控制語句：緊急停止與略過

立即跳出整個迴圈。
(偵測到危險數值，停止所有檢測)

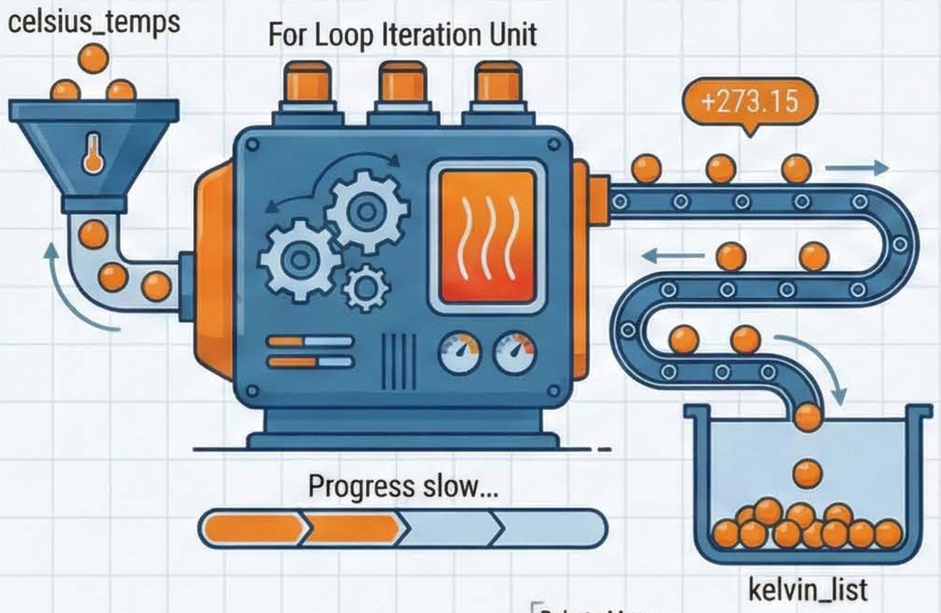
跳過本次迭代，直接進入下一次。
(數據遺失，跳過該筆記錄)

```
for data in sensor_stream:
    if data == 'Error':
        break          # 停止程序 (Stop)
    if data is None:
        continue       # 略過無效數據 (Skip)
    process(data)
```

串列生成式 (List Comprehension)：高速產線優化

這是 Python 特有的語法糖，將傳統的 for 迴圈壓縮成一行，既簡潔又高效。

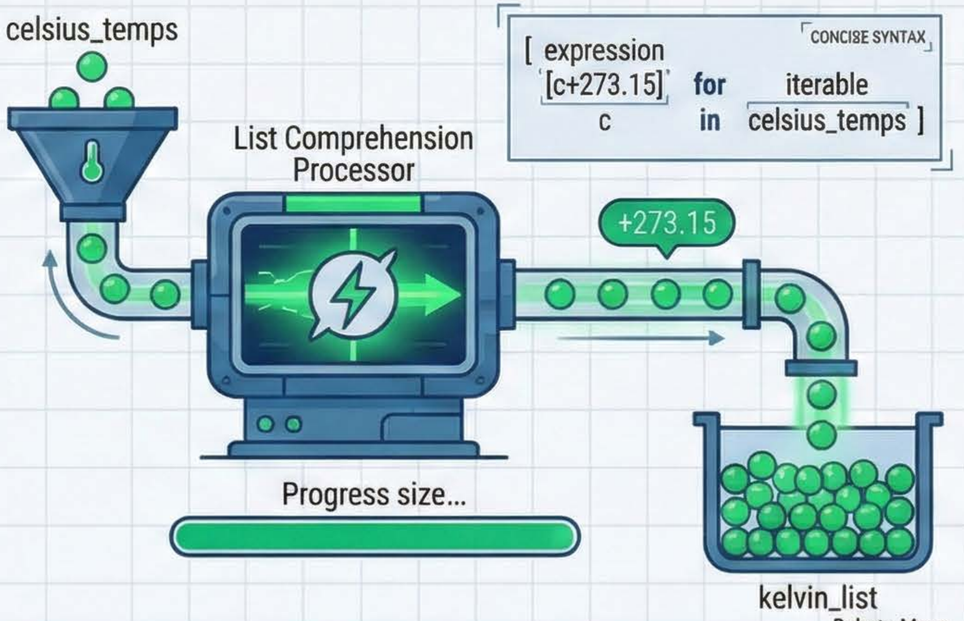
Traditional Approach (傳統寫法)



Roboto Mone

```
kelvin_list = []
for c in celsius_temps:
    kelvin_list.append(c + 273.15)
```

List Comprehension (優化寫法)



Roboto Mone

```
kelvin_list = [c + 273.15 for c in celsius_temps]
```

優勢：減少程式碼行數 (Code Density) 並提升可讀性。

NotebookLM

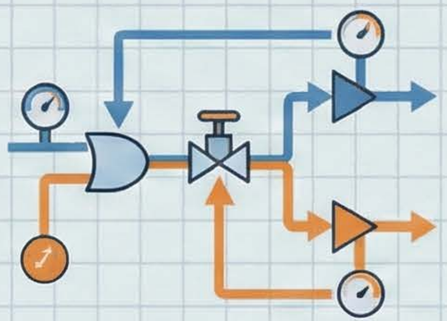
系統診斷總結：Unit 02 Section 04

Conditions (條件語句)

「Reboto Mono」

```
if / elif / else
```

關鍵：縮排 (Indentation) 決定區塊範圍。

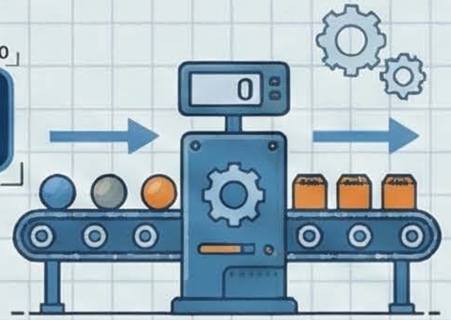


For Loop (定序迴圈)

「Reboto Mono」

```
for item in sequence:
```

用途：處理已知數量的清單或範圍。

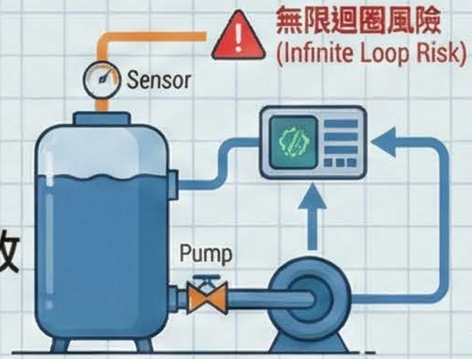


While Loop (條件迴圈)

「Roboto Mono」

```
while condition:
```

用途：持續執行直到狀態改變（需注意無限迴圈）。

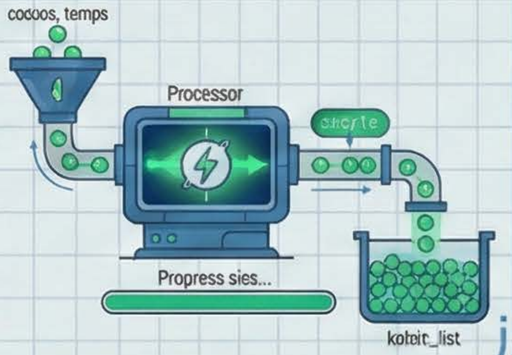


List Comprehension (串列生成式)

「Roboto Mono」

```
[expression for
  item in list]
```

用途：快速建立或轉換資料清單。



下一步：Section 05 函式 (Functions) - 將邏輯封裝成模組。